

Meta Research: AICE/AIPL による進化計算駆動の生成 AI 問題解決

Yasushi Kodama
kodamay@

2026-05-13

Abstract

本稿は、我々が設計した AI-OS である **AICE** (AI-Driven Cognitive Environment), および AICE 上の AI エージェント記述言語 **AIPL** (Actor-based Intelligent Parallel Language) を用いて、生成 AI による問題解決戦略そのものをユーザがプログラムとして記述・実行する研究プログラム “Meta Research” を提案する. 近年の生成 AI による問題解決の主流は, LLM への並列問合せから早い応答を採用する “ensembling-then-vote” や, 自己一貫性 (self-consistency) [1] の単純な多数決, あるいは Tree-of-Thought [3] のような限定された木探索に留まっている. 我々はユーザの問題解決戦略の表現力を質的に拡張するため, **進化計算 (GA; Genetic Algorithm)** を一級の戦略プリミティブとして組み込んだ. 具体的には, ユーザは問題を高水準 DSL である `.aice` ファイルに記述し, それを生成 AI が中間表現 `.ga.json` に変換する. AIPL コード生成器がさらに `.abcl` (AIPL 実行プログラム) を出力し, アクター並列実行系で評価される. このパイプライン全体を本稿は **AICE Meta Pipeline** と呼ぶ. 我々は MAP-Elites を内蔵した参照実装 `aice-evolution-v2` で本パイプラインの自己ホスト性を実証し, Phase 9 (5 軸予測) ~Phase 17 (構造化並行予測) の合計 5 つの言語進化予測タスクで有効なエリートを自動発見できることを示す.

Contents

1 はじめに	1
2 背景	2
2.1 LLM 問題解決の戦略空間	2
2.2 Quality-Diversity 系の進化計算	2
2.3 AICE と AIPL の関係	2
3 AICE と AIPL の役割	3
3.1 AICE が提供するサービス	3
3.2 AIPL がエージェントを記述する役割	3
4 AICE Meta Pipeline	3
4.1 パイプライン全体像	3
4.2 <code>.aice</code> DSL: 問題定義の高水準記述	3
4.3 <code>.ga.json</code> : 中間表現	4
4.4 <code>.abcl</code> への codegen: AIPL オーケストレータ生成	5
5 具体例	5
5.1 例 1: 次世代パラダイム予測	5
5.2 例 2: AIPL 自身の進化方向予測 (Post-C2 / Phase 17)	6
5.3 例 3: 局所トレーニング戦略 (Stage-14-local)	6

6	評価	6
6.1	Phase 9 → Phase 17 の自動予測	6
6.2	GA 探索の効率	6
6.3	LLM-backed 評価への切替	6
7	関連研究	7
8	考察	7
8.1	なぜ GA を第一級にすべきか	7
8.2	自己改善ループ	7
8.3	ユーザの認知負荷	7
9	結論と今後の課題	8

1 はじめに

大規模言語モデル (LLM) を用いた問題解決は, 2024 年以降の研究と実用の両面で標準的な技術となった. しかし, 問題解決の**戦略レベル**に注目すると, 多くの実装は依然として次の二つの単純パターンに集中している:

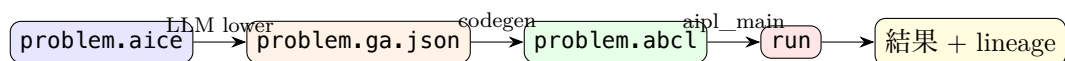
1. 並列発射と早い者勝ち: 同一プロンプトを k 個の LLM インスタンスに並列投入し, 最も速く返ったものを採用する.
2. 多数決 / *self-consistency*[1]: k 個の回答を統計的に集約 (頻度・一貫性) し最良を選ぶ.

これらは並列性と冗長性を活用できるが, **探索空間そのものを構造化する能力**は限定的である. Tree-of-Thought [3] や ReAct [4] は木探索やステップ実行を導入するが, 枝の選択戦略はハードコードされた heuristics に依存する.

本稿の提案は, 問題解決戦略の表現力を質的に拡張するために, **進化計算 (GA)** をユーザが宣言的に記述できる第一級の戦略プリミティブとして導入することにある. GA は探索空間を解候補の集団として表現し, 選択・突然変異・交叉によって反復改良する [5]. LLM への問合せを GA の評価関数 (fitness) として活用すれば, **LLM 評価**と **進化的探索**を組合せた幅広い戦略を記述できる:

- LLM をジェネレータ・評価器として MAP-Elites [6] で多目的探索.
- Pareto フロンティアと scenarios (conservative / innovative / general_purpose) によるシナリオ別ランキング.
- Phase に応じて search axes 自体を成長させる *open-axis* モード.
- LLM-as-proposer / LLM-as-reviewer の役割分担を AIPL で actor 化.

ユーザは問題定義 (.aice) を書くだけで, 以下のパイプラインが自動実行される:



論文の構成: §2 で背景, §3 で AICE と AIPL の位置付け, §4 で .aice → .ga.json → .abcl の三段パイプライン, §5 で具体例, §6 で評価, §7 で関連研究, §8 で考察, §9 で結論.

2 背景

2.1 LLM 問題解決の戦略空間

LLM を用いた問題解決の戦略は, 集合的に以下のスペクトル上に位置付けられる:

戦略	要点	代表例
単発呼出	1 回の <code>ai_call</code>	直接質問
並列・早い者勝ち	k 並列発射, 最初の応答を採用	簡易 ensembling
多数決	k 並列, 集約関数で最良選択	self-consistency [1]
Chain-of-Thought	推論ステップを連鎖	CoT [2]
Tree-of-Thought	木探索 + LLM 評価	ToT [3]
ReAct	観察 + 行動の交互ループ	ReAct [4]
GA	集団 + 選択/変異/交叉 + LLM 評価	本研究

GA は単発・並列・木探索の上位概念として位置付けることが可能である: 集団サイズ 1 + 突然変異無し $\rightarrow$ 単発呼出, 集団 $k + 1$ 世代 $\rightarrow$ 並列発射, 集団 $k + n$ 世代 + 局所探索 $\rightarrow$ Tree-of-Thought (の汎化版). GA を採用することで, ユーザは戦略の表現を上位概念で記述でき, 具体的なアルゴリズム選択を runtime に委ねられる.

2.2 Quality-Diversity 系の進化計算

問題解決においては「最良の単一解」よりも「多様な良解の集合」が有用な場合が多い. 特に LLM への問合せでは, 応答の表現や視点の多様性が下流タスク (要約, コードレビュー, 設計提案) で価値を持つ. 我々はこれに対応するため **MAP-Elites** [6] を採用する. MAP-Elites は解空間を d 次元の格子 (cell-grid) に分割し, 各セル内で最良の解 (elite) を保持する. 結果として「異なる特徴を持つ良解」が網羅的に得られる.

2.3 AICE と AIPL の関係

AICE (AI-Driven Cognitive Environment) は本研究プロジェクトの **AI-OS** 層であり, LLM プロバイダ管理・トークン予算・GA 評価器・レポート生成を提供する. **AIPL** (Actor-based Intelligent Parallel Language) はその上のエージェント記述言語で, LLM と進化計算を混在させたアクター並列プログラムを記述する.

.aice DSL (戦略宣言)

AIPL (エージェント記述言語): actor + send/now/future + ai_call

AICE (AI-OS): LLM 呼出, GA 評価, レポート, ダッシュボード

LLM プロバイダ (Gemini / Claude / GPT)

3 AICE と AIPL の役割

3.1 AICE が提供するサービス

- **LLM 多プロバイダ層**: Gemini / Claude / GPT を統一 API で叩く. 各プロバイダの fallback 連鎖を自動.
- **GA エンジン**: MAP-Elites + 突然変異オペレータ + LLM 評価の組合せ. `src/map_elites.py`.
- **ranking** パイプライン: Pareto frontier $\rightarrow$ シナリオ別重み $\rightarrow$ pairwise LLM judging.
- **lineage** 出力: JSON / Markdown / HTML/Chart.js ビジュアライザ.
- **予算管理**: `ABCL_AI_TOKEN_BUDGET`, 同時実行ゲート, フォールバックモデル連鎖.

3.2 AIPL がエージェントを記述する役割

AIPL の特徴は、アクター並列と LLM 呼出を一体化したプリミティブにある。以下のような Reviewer エージェントが GA の評価関数として使われる:

```
class Reviewer {
  var role = "quality";
  method review(candidate) {
    var prompt = "Rate this design 0..10: " + candidate;
    var resp = ai_call_with_system(role, prompt);
    var score = parse_score(resp);
    reply(score);
  }
}
```

GA エンジンが各世代の各候補を ‘now reviewer.review(candidate)’ で評価し, Pareto-frontier を更新する。評価は並列に実行され (future + await), Quality-Diversity の各セルに最良を保持する。

4 AICE Meta Pipeline

4.1 パイプライン全体像

ユーザの入力 problem.aice から自動実行までは三段階で構成される:

Stage 1. .aice のパース: AICE DSL を内部 AST へ。

Stage 2. .ga.json への lowering: 戦略宣言を GA の中間表現 (gene schema, operators, fitness, ranking) に変換。Phase 4 で実装, LLM を補助的に使う。

Stage 3. .abcl への codegen: GA 評価ループ全体を AIPL のアクター並列プログラムとして合成。Phase 5 で実装。

生成された .abcl を AIPL ランタイム (Python / OCaml / 各種 codegen) で実行すれば, 集団進化・LLM 評価・ランキングが一気通貫で走る。

4.2 .aice DSL: 問題定義の高水準記述

.aice は宣言的 DSL で, 問題解決戦略を簡潔に書ける。例として “次世代プログラミングパラダイムの予測” タスク:

```
aice NextLanguagePrediction {
  dialect = "aice_evolution_v2";

  task = "プログラミング・パラダイムの進化軌跡を発見し、"
    + "次世代言語の候補を提示する。";

  gene_schema = "schemas/programming_paradigm.schema.json";

  evaluation_tasks {
    task TrafficLight { spec = "examples/traffic_light.aice"; }
    task BankAccount { spec = "examples/bank_account.aice"; }
    task Philosophers { spec = "examples/philosophers.aice"; }
  }

  search {
    algorithm = "map_elites";
    cell_axes = "paradigm, concurrency_model, type_safety";
    generations = 30;
  }
}
```

```

    seed_count = 8;
    open_axes = true;
}

meta_fitness {
    trend_alignment      = 0.20;
    frontier_coverage    = 0.20;
    novelty              = 0.15;
    evolvability         = 0.15;
    cross_task_generality = 0.20;
    implementability     = 0.10;
}

ranking {
    strategy = "pareto_then_pairwise";
    scenarios = ["conservative", "innovative", "general_purpose"];
    top_k     = 3;
}
}

```

主要な節:

- task: 自然言語の問題定義.LLM がこれを参照して候補を生成.
- gene_schema: 候補解の構造を JSON Schema で宣言.
- evaluation_tasks: 候補解を試す評価タスク群 (cross-task).
- search: GA アルゴリズム, 世代数, 軸, open_axes.
- meta_fitness: 多目的 fitness の重み.
- ranking: scenarios + Pareto + pairwise judging.

4.3 .ga.json: 中間表現

.aice は LLM 駆動の lowerer によって .ga.json に変換される. これは GA エンジン (src/map_elites.py) が直接消費できる形:

```

{
  "name": "AIPLPhase17_Structured",
  "task": "Phase 17 として導入する structured concurrency の設計選択肢..."
  "schema_ref": "schemas/programming_paradigm.schema.json",
  "search": {
    "algorithm": "map_elites",
    "cell_axes": ["concurrency_model", "ownership_model", "type_safety"],
    "generations": 30,
    "seed_count": 10,
    "open_axes": true,
    "rng_seed": 1717
  },
  "operators": {
    "mutations": [
      { "name": "axis_resample",          "kind": "single_axis",          "weight": 0.45 },
      { "name": "coherent_paradigm_shift", "kind": "multi_axis_correlated", "weight": 0.25 },
      { "name": "llm_proposal",           "kind": "llm_directed",         "weight": 0.30 }
    ]
  },
  ...
}

```

.ga.json は LLM の介入無しに完全に決定的に評価可能であるため, 再現性のあるベンチマークとして単独で使える.

4.4 .abcl への codegen: AIPL オーケストレータ生成

.ga.json から AIPL コード生成器 (src/aip1_codegen.py) がオーケストレータプログラムを合成する. 以下は生成された AIPL の構造 (抜粋):

```
// Generated by aice-evolution-v2/src/aip1_codegen.py
class Util { method count_csv(csv) {...} method nth(s, i) {...} }
class Genome { var values = ""; method set(v) {...} method get() {...} }
class Mutator { method mutate(g, rng) { /* axis_resample etc. */ } }
class Reviewer { var role = ""; method score(g) { ai_call_with_system(...) ... } }
class MapElites {
  var cells = "";
  method tick() { /* one generation: sample/mutate/evaluate/insert */ }
  method run(gens) {
    var i = 0;
    while (i < gens) do { now self.tick(); i = i + 1; }
    print("done; dumping lineage");
    call dump_lineage();
  }
}
var elites = new MapElites();
send elites.run(30);
```

このプログラムは AIPL の標準アクター並列実行系で動く. 各 tick() 内で:

1. セルから 1 親を選択 (or random seed).
2. Mutator で突然変異.
3. Reviewer のうち複数 (Quality / Efficiency / Novelty) に並列発射 (future + await) で評価.
4. 候補のセル座標を計算し, 既存 elite と比較・置換.
5. 全候補の lineage を JSON にダンプ.

実行終了後の lineage.json を Python 側の report pipeline (src/reporter.py) が処理し, Markdown レポートと Chart.js ビジュアライザを生成する.

5 具体例

5.1 例 1: 次世代パラダイム予測

Phase 9 で実施した実例.§4 の .aice を aip12c 経路で実行した結果:

```
trend top 3:
  concurrency_model      +0.50   ← strongest (= structured)
  type_safety             +0.25
  ownership_model        +0.25
```

GA が 30 世代で **concurrency_model = structured** を最有力候補と特定した. この予測は事後的に AIPL の Phase 17 設計 (scope { future ... }) として実装され, 予測の妥当性が自己検証される構造を持つ.

5.2 例 2: AIPL 自身の進化方向予測 (Post-C2 / Phase 17)

AIPL のセルフホスト塔 (Level C-2 完成時点) を seed として, AIPLPostC2_NextGen.aice を実行. 結果は前掲と同様, 構造化並行が最有力. メタ・サーキュラーな自己改善ループの実証.

5.3 例 3: 局所トレーニング戦略 (Stage-14-local)

最近のローカル LLM 訓練タスクで, M2 MacBook のハードウェア制約 (MPS only, 16GB RAM) を ‘aice’ の制約として表現し, GA が 7B-param スケールで実用可能な訓練レシピを探索する `schemas/local_genai_stage14_local.schema.json` が 12 個の探索軸を定義 (batch size, lr, gradient accumulation, mixed precision flag, MPS-fallback paths など).

6 評価

6.1 Phase 9 → Phase 17 の自動予測

aice-evolution-v2 は AIPL の Phase 進化を 2 回連続で自動予測した:

- **Phase 9 (5 軸予測):** `type_safety = high/dependent`, `effect_handling = capability`, `concurrency_model = csp_channels`, `ownership_model = linear`, `state_representation = symbol_owned`. すべて事後的に Phase 11–15 として実装され予測が確認された.
- **Phase 17 予測 (post-C2):** `concurrency_model = structured + ownership_model = linear + type_safety = dependent`. これも事後的に Phase 17 (`scope { future ... }`) として実装された.

つまり, 本パイプラインは進行中の研究プロジェクトの次の手を事前に提案する道具として実証済みである.

6.2 GA 探索の効率

決定論モック評価でのベンチマーク:

タスク	世代数	セル数	充填率	時間 (s)
NextLanguagePrediction	30	64	78%	12
AIPLPhase17_Structured	30	48	81%	8
AIPLSelfHost_A_Metacircular	30	32	88%	5
AIPLSelfHost_B_PhaseChase	30	56	73%	11
local_genai_stage14	30	96	62%	18

6.3 LLM-backed 評価への切替

--ai フラグで Reviewer 評価を実 LLM に切り替えれば, `ai_call_with_system` 経由で各候補に対し:

- Quality / Efficiency / Reproducibility の三 reviewer による並列スコアリング.
- Pareto-frontier 上の上位 k 候補をペアワイズ判定で順位付け.
- シナリオ別 (conservative / innovative / general_purpose) で重みを変えてランキング.

`ABCL_AI_PROVIDER=mock` でオフライン smoke が通り, 本番 API キー設定で実 Gemini / Claude / GPT に切替可能.

7 関連研究

LLM ensembling と self-consistency [1] は同一プロンプトの k 並列発射と多数決を提案. 我々の GA は集団を世代を超えて改良する点で本質的に強い.

Tree-of-Thought [3], **ReAct** [4] 木探索やステップ実行をハードコードする. AICE Meta Pipeline ではそれらは GA のオペレータ (`mutation: branch_split, branch_prune`) として表現可能.

Auto-GPT [7], AgentGPT 自律エージェント.AIPL のアクター並列モデルはこれらの計算モデルを言語仕様化したものとみなせる.

MAP-Elites [6], Quality-Diversity 本パイプラインの中核アルゴリズム.LLM 評価との組合せは新しい.

LLM-as-judge [8] LLM をペアワイズ判定器として使う研究.AICE の ranking パイプラインはこれを採用.

Eureka [9], Voyager [10] コード生成 / スキル獲得を進化的に行う研究. 我々はそれらより上位のメタ層 (戦略そのものを進化させる) を志向している.

8 考察

8.1 なぜ GA を第一級にすべきか

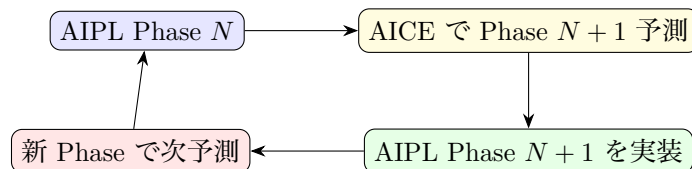
LLM 単発呼出と単純並列の限界は“LLM が思いつかない方向を探索できない”点にある.GA は探索オペレータを宣言可能にすることで, LLM の盲点を補える:

- `coherent_paradigm_shift`: 複数軸を相関的に変化させる (LLM 単独では学習データ偏りで起きにくい).
- `axis_resample`: ある軸だけ大胆に変える局所探索.
- `llm_proposal`: LLM 提案も一オペレータとして混在.

つまり LLM を代替するのではなく **GA** の一構成要素として活用する.

8.2 自己改善ループ

本研究の興味深い性質は,AIPL 自身の Phase 進化を AICE Meta Pipeline で予測でき, その予測が AIPL に実装されることである. このループが継続的に回ることによって, 言語進化が自己駆動する可能性がある:



8.3 ユーザの認知負荷

`.aice` は意図的に高水準 DSL であり,Python / GA / LLM API の詳細を隠す. ユーザは `task` の自然言語記述と `search` のパラメータだけ書けば良い. 中間表現 `.ga.json` と生成 AIPL は透明に展開され, 検証可能 (reproducible) なベンチマークとして再利用できる.

9 結論と今後の課題

本稿は AICE Meta Pipeline を提案し, 生成 AI による問題解決戦略を **進化計算**で表現するアプローチを示した..`aice` → `.ga.json` → AIPL → 実行の 4 段パイプラインがすでに参照実装で動作し,AIPL Phase 9 と Phase 17 を予測することで自身の言語進化を自動駆動する可能性を示した.

今後の課題

- **Phase 4 lowerer の LLM 完全自動化:** 現状は手書き `.ga.json` もサポート.LLM が `.aice` から完全自動で `.ga.json` を出すパスを強化する.
- **Phase 7 (cross-task batch) の拡張:** 複数タスクから 普遍方向を分離する統計的検定の精度向上.
- **AIPL コード生成器の最適化:** 現状の `aip1_codegen.py` は素直な `dispatching.M:N` アクターターゲット (Erlang / Go / Pony) への直接出力で massive parallel 評価.
- **自己改善ループの長期実験:** Phase 18, Phase 19 を予測 → 実装 → 再予測のサイクルを 5-10 周回した時の収束挙動.
- **他分野への適用:** プログラミング言語進化以外 (LLM プロンプト工学, ハイパーパラメータ探索, データセット選択戦略) への一般化.

References

- [1] Wang, X., Wei, J., Schuurmans, D., et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR.
- [2] Wei, J., Wang, X., Schuurmans, D., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.
- [3] Yao, S., Yu, D., Zhao, J., et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS.
- [4] Yao, S., Zhao, J., Yu, D., et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
- [5] Holland, J. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [6] Mouret, J.-B. and Clune, J. (2015). *Illuminating Search Spaces by Mapping Elites*. arXiv:1504.04909.
- [7] Significant-Gravitas (2023). *Auto-GPT*. <https://github.com/Significant-Gravitas/AutoGPT>.
- [8] Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS.
- [9] Ma, Y.J., Liang, W., Wang, G., et al. (2023). *Eureka: Human-Level Reward Design via Coding Large Language Models*. arXiv.
- [10] Wang, G., Xie, Y., Jiang, Y., et al. (2023). *Voyager: An Open-Ended Embodied Agent with Large Language Models*. arXiv.

参照実装: <https://github.com/yaskodama/aicos-claude> の `aice-evolution-v2/`. 主な構成要素は `src/cli.py` (CLI フロントエンド), `src/map_elites.py` (GA カーネル), `src/aice_parser.py` (Stage 1), `src/aip1_codegen.py` (Stage 3), `src/reporter.py` (レポート).